

◆ GUIA COMPLETO DE IMPLEMENTAÇÃO ◆

CONSTRUA O JARVIS

COM CLAUDE CODE – DO ZERO AO DEPLOY

Você digita o comando. Claude executa tudo.

Este guia mostra exatamente **o que digitar** em cada etapa para ter seu próprio JARVIS com voz, agentes autônomos e memória permanente rodando em produção — sem escrever uma linha de código.

Claude Code CLI

Next.js + Supabase

OpenAI Realtime

Deploy Vercel

21 Agentes IA

◆ A FILOSOFIA DESTA GUIA ◆

Você não precisa saber programar.
Você só precisa saber **o que pedir**.

01

VOCÊ LEMBRA O COMANDO

Cada passo tem um
prompt pronto para
copiar e colar

02

CLAUDE EXECUTA TUDO

Cria arquivos, instala
dependências, configura
APIs

03

VOCÊ SÓ APROVA

Veja o resultado,
confirme, siga para o
próximo passo

O que você vai construir

PARTE 0 – ANTES DE COMEÇAR (15 minutos)

- 01 Instalar o Claude Code no seu computador
- 02 Criar as contas gratuitas necessárias

PARTE 1 – CRIANDO O PROJETO (30 minutos)

- 03 Comando: criar o projeto JARVIS do zero
- 04 Comando: configurar banco de dados Supabase
- 05 Comando: configurar variáveis de ambiente

PARTE 2 – CONSTRUINDO O JARVIS (2-4 horas)

- 06 Comando: interface visual estilo Iron Man
- 07 Comando: sistema de chat com Claude
- 08 Comando: sistema de voz com wake word
- 09 Comando: memória permanente
- 10 Comando: 21 agentes autônomos
- 11 Comando: painel DOCS com download ZIP
- 12 Comando: worker autônomo de ideias
- 13 Comando: tela de senha Iron Man

PARTE 3 – DEPLOY EM PRODUÇÃO (15 minutos)

- 14 Comando: publicar no GitHub
- 15 Comando: deploy na Vercel (gratuito)

PARTE 4 – USANDO O JARVIS

- 16 Como falar com o JARVIS (wake word + voz)
- 17 Como pedir projetos e arquivos
- 18 Checklist final + comandos de referência

PARTE 5 – AIOS CORE: INSTALAÇÃO E CAPACIDADES

- 19 O que é o AIOS CORE e para que serve

20 Comando: instalar o AIOS CORE do zero

21 Os 6 agentes do AIOS CORE — o que cada um faz

22 O que você pode construir com o AIOS CORE

23 Comando: expandir com novos agentes customizados

24 ORÁCULO + CÉREBRO — inteligência contínua

Instalar o Claude Code

Única instalação manual necessária – leva 5 minutos

Claude Code é a ferramenta que você vai usar para construir tudo. É um programa que roda no terminal do seu computador e executa comandos por você.

Pré-requisitos

1

Node.js instalado

Baixe em

nodejs.org

→ versão LTS (botão verde). Instale normalmente como qualquer programa Windows.

2

Conta na Anthropic criada

Acesse

console.anthropic.com

→ crie sua conta → gere uma API Key em "API Keys"

Instalar o Claude Code

Abra o **Prompt de Comando** (CMD) ou **PowerShell** e execute:

```
> npm install -g @anthropic-ai/claude-code  
> claude
```

```
# Na primeira execução, vai pedir para fazer login  
# Pressione Enter, abra o link no browser e autorize
```

► COMO SABER QUE FUNCIONOU

Você verá o prompt `claude>` ou uma interface de chat no terminal. Digite **"olá"** e Claude responde. Está pronto.

Criar as Contas Necessárias

Todas gratuitas – anote as chaves em um bloco de notas

SERVIÇO	ONDE CRIAR	O QUE PEGAR	CUSTO
Anthropic	console.anthropic.com	API Key (sk-ant-...)	~\$10/mês uso pessoal
OpenAI	platform.openai.com	API Key (sk-proj-...)	~\$5/mês voz
Supabase	supabase.com	URL do projeto + Anon Key	Gratuito
Vercel	vercel.com	Login com GitHub	Gratuito
GitHub	github.com	Criar conta + repositório	Gratuito

⚠ ANOTE TUDO AGORA

Abra um bloco de notas e cole todas as chaves antes de continuar. Você vai precisar delas no Passo 05. **Nunca compartilhe essas chaves com ninguém.**

Criando o projeto no Supabase

1

Acesse supabase.com → New Project

Escolha a região: South America (São Paulo). Defina um nome e senha para o banco.

2

Copie as credenciais

No projeto criado: Settings → API → copie a

Project URL

e a

anon public key



Abra o SQL Editor

No menu lateral: SQL Editor → New Query. Deixe aberto para o Passo 04.

Criar o Projeto JARVIS

Digite este comando no Claude Code → ele cria toda a estrutura

◆ COMO USAR ESTES COMANDOS

Abra o terminal na pasta onde quer criar o projeto. Execute `claude` para entrar no Claude Code. Copie o texto da caixa verde e cole no chat do Claude. Ele executará tudo automaticamente.

► COMANDO 01 - Cole no Claude Code

CLAUDE CODE - COLE ESTE PROMPT

Crie um novo projeto Next.js 15 chamado "meu-jarvis" com TypeScript, Tailwind CSS 3.4, ESLint e App Router. Use o diretório `src/`. Depois instale as dependências: `@anthropic-ai/sdk`, `openai`, `jszip`, `@supabase/supabase-js`. Inicialize também um repositório git no projeto. Me mostre a estrutura de pastas criada no final.

O QUE CLAUDE VAI CRIAR

- ✓ Pasta `meu-jarvis/` com Next.js 15 configurado
- ✓ TypeScript + Tailwind CSS + ESLint prontos
- ✓ Dependências instaladas (Anthropic, OpenAI, JSZip, Supabase)
- ✓ Repositório git inicializado
- ✓ Estrutura `src/app/ src/components/ src/hooks/ src/lib/ src/types/`

⚠ APÓS CLAUDE TERMINAR

Execute `cd meu-jarvis` no terminal para entrar na pasta do projeto. Todos os próximos comandos do Claude Code devem ser executados de dentro desta pasta.

Configurar o Banco de Dados

Claude cria todo o schema SQL – você só cola no Supabase

► COMANDO 02 – Cole no Claude Code

CLAUDE CODE – COLE ESTE PROMPT

```
Crie o arquivo docs/supabase-schema.sql com o schema completo para o sistema JARVIS. Precisa das tabelas: jarvis_memory (id uuid, role text, content text, session_id text, importance int, created_at timestampz), user_facts (id uuid, fact text, category text, importance int, source text, created_at timestampz), agent_knowledge (id uuid, agent_id text, skill_name text, content text, quality int, created_at timestampz), jarvis_files (id uuid, path text, content text, project_name text, session_id text, created_at timestampz), oraculo_knowledge (id uuid, topic text, content text, source text, quality int, created_at timestampz). Adicione índices de performance e Row Level Security com policy allow_all para todas as tabelas. Mostre o conteúdo completo do arquivo criado.
```

APÓS CLAUDE CRIAR O ARQUIVO

- ✓ Copie TODO o conteúdo do arquivo docs/supabase-schema.sql
- ✓ Abra supabase.com → seu projeto → SQL Editor
- ✓ Cole o SQL na caixa → clique em "Run"
- ✓ Você verá "Success" para cada tabela criada

Configurar as Chaves de API

Use as chaves que você anotou no Passo 02

► COMANDO 03 - Cole no Claude Code (substitua os valores)

CLAUDE CODE - COLE ESTE PROMPT

Crie o arquivo `.env.local` na raiz do projeto com estas variáveis de ambiente:

```
ANTHROPIC_API_KEY=SUA_CHAVE_ANTHROPIC_AQUI
OPENAI_API_KEY=SUA_CHAVE_OPENAI_AQUI
NEXT_PUBLIC_SUPABASE_URL=SUA_URL_SUPABASE_AQUI
NEXT_PUBLIC_SUPABASE_ANON_KEY=SUA_ANON_KEY_SUPABASE_AQUI
NEXT_PUBLIC_WAKE_WORD=jarvis
NEXT_PUBLIC_JARVIS_VOICE=alloy
NEXT_PUBLIC_REALTIME_MODEL=gpt-4o-realtime-preview
```

Depois adicione `.env.local` ao `.gitignore` para não subir as chaves para o GitHub.

O QUE CLAUDE FAZ

- ✓ Cria o arquivo `.env.local` com suas chaves
- ✓ Verifica que `.env.local` está no `.gitignore`
- ✓ Confirma que as variáveis estão configuradas corretamente

Interface Visual – Cockpit Iron Man

O visual futurista completo com animações e HUD

► COMANDO 04 – Cole no Claude Code

CLAUDE CODE – COLE ESTE PROMPT

Crie a interface visual completa do JARVIS estilo Iron Man / Tony Stark com as seguintes especificações:

CORES: fundo #020609, azul principal #00d4ff, verde #00ff88, cyan #00ffff, roxo #a78bfa

LAYOUT: 3 colunas (painel esquerdo 320px + centro flex + painel direito 320px) + status bar no topo 40px + terminal 200px na base

COMPONENTES NECESSÁRIOS:

- src/components/cockpit/JarvisCockpit.tsx (componente raiz)
- src/components/cockpit/StatusBar.tsx (barra de status com relógio, indicadores VOZ/CLAUDE/AGENTES)
- src/components/cockpit/HexGrid.tsx (fundo com hexágonos animados via SVG)
- src/components/cockpit/CentralOrb.tsx (orbe central animada que reage ao áudio via Web Audio API analyser)
- src/app/page.tsx (importa JarvisCockpit)
- src/app/layout.tsx (fontes: Orbitron + Share Tech Mono via Google Fonts)
- src/app/globals.css (reset + variáveis CSS + animações pulse/float)
- next.config.ts (headers COEP/COOP para Web Audio API)

Use Tailwind CSS para layout. Todos os textos de status em fonte monospace. Adicione scan lines e vignette como overlays CSS. Os cantos devem ter brackets HUD decorativos. Rode npm run dev e mostre se compilou sem erros.

RESULTADO ESPERADO

- ✓ Interface escura estilo Iron Man acessível em localhost:3000
- ✓ Fundo com hexágonos animados sutilmente
- ✓ Orbe central azul/cyan no meio
- ✓ Status bar no topo com relógio em tempo real

✓ Zero erros TypeScript no terminal

Sistema de Chat com Claude

JARVIS responde via streaming em tempo real

► COMANDO 05 - Cole no Claude Code

CLAUDE CODE - COLE ESTE PROMPT

Implemente o sistema de chat com Claude Sonnet 4.6 via Server-Sent Events (SSE) com streaming em tempo real. Crie:

1. `src/app/api/jarvis-chat/route.ts` - endpoint POST que usa `@anthropic-ai/sdk` com `stream()`, lê o system prompt de `src/lib/jarvisPrompt.ts`, processa `tool_use` para ferramentas (`write_file`, `save_project`, `remember_fact`), e envia cada token como evento SSE `"data: TOKEN\n\n"`
2. `src/lib/jarvisPrompt.ts` - system prompt completo do JARVIS: personalidade Iron Man britânica, sempre chama o usuário de "Sir", tom conciso e direto, documenta as ferramentas disponíveis
3. `src/hooks/useJarvisChat.ts` - hook React que conecta ao SSE, acumula tokens, gerencia estados: `'idle'|'thinking'|'streaming'|'done'|'error'`, expõe `messages[], status, sendMessage(), clearChat()`
4. `src/components/cockpit/ChatPanel.tsx` - painel de chat com bolhas de mensagem estilo HUD, indicador de digitação animado, input com Enter para enviar, scroll automático para o fundo
5. Integre `ChatPanel` no `JarvisCockpit` na coluna esquerda

Teste enviando "JARVIS, qual é seu status?" e confirme que a resposta aparece em streaming.

RESULTADO ESPERADO

- ✓ Painel de chat funcional na coluna esquerda
- ✓ Mensagens aparecem token por token (streaming visível)
- ✓ JARVIS responde com personalidade britânica chamando de "Sir"

✓ Estados: "PENSANDO..." → texto aparecendo → "PRONTO"

Sistema de Voz com Wake Word

"JARVIS" ativa a voz - ele só responde quando chamado pelo nome

► COMANDO 06 - Cole no Claude Code

CLAUDE CODE - COLE ESTE PROMPT

Implemente o sistema de voz completo do JARVIS com OpenAI Realtime API. Crie:

1. `src/app/api/realtime-token/route.ts` - gera token efêmero POST para `https://api.openai.com/v1/realtime/sessions` usando `OPENAI_API_KEY`
2. `src/lib/realtimeClient.ts` - cliente WebSocket que conecta ao endpoint `wss://api.openai.com/v1/realtime?model=gpt-4o-realtime-preview`, emite eventos tipados
3. `src/hooks/useAudioCapture.ts` - captura microfone via `getUserMedia`, converte para PCM16 em chunks de 100ms via `ScriptProcessor`, emite base64
4. `src/hooks/useAudioPlayback.ts` - reproduz chunks de áudio PCM16 recebidos via `AudioContext`, controla barge-in (parar quando usuário fala)
5. `src/hooks/useWakeWord.ts` - usa Web Speech API (`SpeechRecognition`) em modo contínuo, detecta "jarvis" no transcript, captura o comando depois do wake word, IMPORTANTE: `create_response` deve ser false por padrão (JARVIS não responde a qualquer fala, só quando chamado)
6. `src/hooks/useRealtimeSession.ts` - orquestra tudo: conecta WebSocket, configura `session.update` com `turn_detection` `create_response: false`, gerencia estados `'idle'|'connecting'|'active'|'thinking'|'speaking'`, implementa conversation mode de 30s após wake word
7. Integre no JarvisCockpit: botão "🔊 MIC ATIVO / 🔇 MIC MUDO" no centro, orbe que reage ao áudio, indicador de estado de voz

Confirme que ao dizer "JARVIS, qual a hora?" a voz é detectada e JARVIS responde.

RESULTADO ESPERADO

- ✓ Botão "🔊 MIC ATIVO" visível no centro do cockpit
- ✓ Orbe anima ao falar (reage ao volume do microfone)
- ✓ Diga "JARVIS, boa tarde" → JARVIS responde com voz
- ✓ Fale sem dizer "JARVIS" → silêncio total (passivo)
- ✓ Após wake word, 30s de conversa livre sem precisar chamar novamente

⚠ IMPORTANTE - USE CHROME

O wake word usa Web Speech API que só funciona no Google Chrome. No Firefox ou Edge pode não funcionar. Abra localhost:3000 no Chrome e permita o microfone quando solicitado.

Memória Permanente

JARVIS lembra de tudo entre sessões via Supabase

► COMANDO 07 – Cole no Claude Code

CLAUDE CODE – COLE ESTE PROMPT

Implemente o sistema de memória permanente do JARVIS usando Supabase. Crie:

1. `src/lib/supabase.ts` – cliente Supabase usando `NEXT_PUBLIC_SUPABASE_URL` e `NEXT_PUBLIC_SUPABASE_ANON_KEY`, exporta `supabase` e `isSupabaseEnabled`
 2. `src/lib/jarvisMemory.ts` – funções: `saveMessage(role, content, sessionId)`, `searchMemory(query, limit)`, `saveUserFact(content, category, importance, source)`, `getRecentContext(sessionId, limit)`
 3. `src/app/api/jarvis-memory/route.ts` – endpoint POST que executa ferramentas de memória: `remember_fact` (salva em `user_facts`), `search_memory` (busca em `jarvis_memory` + `user_facts`), `write_file` (salva em `jarvis_files`), `save_project` (salva múltiplos arquivos em `jarvis_files`)
 4. Adicione as ferramentas `remember_fact`, `search_memory`, `write_file`, `save_project` à sessão OpenAI Realtime (no `session.update` do `useRealtimeSession`)
 5. No `useRealtimeSession`, quando `response.output_item.done` com `type function_call`, faça `fetch` para `/api/jarvis-memory` e envie o resultado de volta como `function_call_output`
 6. Integre memória no `jarvis-chat`: antes de responder, busque os últimos 5 fatos relevantes do Supabase e injete no contexto do Claude
- Teste: diga "JARVIS, guarde que meu nome é [seu nome]", recarregue a página, diga "JARVIS, qual é meu nome?" – ele deve lembrar.

RESULTADO ESPERADO

- ✓ JARVIS lembra fatos após recarregar a página
- ✓ Ferramenta `remember_fact` salva no Supabase (visível no dashboard)

✓ Ferramenta `search_memory` recupera contexto relevante

✓ Arquivos criados aparecem na tabela `jarvis_files`

21 Agentes Autônomos

AIOS Core (6) + Fábrica Rentável (15) – orquestrados pelo JARVIS

► COMANDO 08 – Cole no Claude Code

CLAUDE CODE – COLE ESTE PROMPT

Implemente o sistema completo de 21 agentes autônomos do AIOS. Crie:

1. `src/types/agents.ts` – tipos: `AgentId`, `AgentState` (`id`, `name`, `status`, `currentTask`, `lastResult`, `skills`), `DelegateCommand`

2. `src/app/api/agent-execute/route.ts` – endpoint POST que recebe `{agent, task, context, priority}`, tem um dicionário `AGENTS` com system prompts para todos os 21 agentes (`@analyst @developer @researcher @writer @ux-design-expert @manager @ideias-nichos @criador-conteudo @produtor-cursos @designer @empacotador @cortes-virais @gestor-contas @trafego-organico @vendas @relacionamento @auditor @analista-metricas @automacao-tecnica @financeiro-pix @melhoria-continua`), executa via `claude-sonnet-4-6` com `max_tokens 8192`

3. `src/lib/agentRouter.ts` – `parseDelegations(text)`: extrai padrões `[DELEGATE: {"agent":"...","task":"...","priority":"...","context":"..."}]` do texto do JARVIS, `parseExecutions(text)`: extrai `[EXECUTE: {"command":"..."}]`, `stripProtocols(text)`: remove os padrões do texto para exibição limpa

4. `src/hooks/useAgentOrchestrator.ts` – gerencia `agentStates[]`, `commandLog[]`, `handleDelegations(commands)` que chama `/api/agent-execute` em paralelo para cada agente, `handleExecutions(commands)` para execuções de terminal

5. `src/components/cockpit/AgentSquadPanel.tsx` – painel direito mostrando todos os 21 agentes em grid com status (ativo/ocioso), tarefa atual, última entrega

6. Atualize `jarvisPrompt.ts` com o sistema completo de dois times e o protocolo `DELEGATE`

Teste: no chat, peça "JARVIS, delegue para o `@analyst` uma análise de mercado de SaaS" e veja o agente aparecer como ativo no painel.

RESULTADO ESPERADO

- ✓ Painel de 21 agentes na coluna direita com status em tempo real
- ✓ JARVIS delega automaticamente quando recebe pedidos complexos
- ✓ Agentes ficam "ATIVOS" durante execução e voltam a "OCIOSO"
- ✓ Resultados dos agentes aparecem no chat

Painel DOCS com Download ZIP

Todos os arquivos criados pelo JARVIS disponíveis para baixar

► COMANDO 09 - Cole no Claude Code

CLAUDE CODE - COLE ESTE PROMPT

Implemente o painel DOCS completo com sistema de download. Crie:

1. `src/hooks/useJarvisDeliveries.ts` - polling a cada 5s no Supabase buscando `jarvis_files` ordenado por `created_at` desc, limit 50, exporta `files[], loading, clear()`
2. `src/components/cockpit/DeliveriesPanel.tsx` - painel com:
 - Header com botão "↓ BAIXAR TUDO" (verde sólido, sempre visível no topo)
 - ProjectCard: para arquivos com `project_name`, header com botão "↓ ZIP" verde sólido, lista de arquivos internos cada um com botão "📄 COPIAR" e "↓ BAIXAR"
 - LooseFileCard: para arquivos sem projeto, header com botões "📄 COPIAR" e "↓ BAIXAR" em verde sólido com texto (não só ícone)
 - Download individual usa `document.body.appendChild(a) + a.click() + removeChild` para compatibilidade
 - Download ZIP usa biblioteca `jszip`, inclui `HOW-TO-RUN.txt` automático detectando Python/Node/HTML
 - Todos os botões de download DENTRO do header row (nunca em div separada abaixo)
3. Adicione aba DOCS na coluna direita (tabs AGENTES | DOCS)
4. Auto-switch para aba DOCS quando novo arquivo chega

Instale `jszip` se não estiver instalado. Verifique que os botões de download funcionam criando um arquivo de teste.

RESULTADO ESPERADO

- ✓ Aba DOCS aparece com badge contando arquivos novos
- ✓ Botões verdes "↓ BAIXAR" visíveis em cada arquivo
- ✓ ZIP baixa com `HOW-TO-RUN.txt` incluído

✓ Auto-switch para DCS quando JARVIS entrega arquivo

Worker Autônomo de Ideias

JARVIS gera briefs de MVP automaticamente a cada ciclo

► COMANDO 10 – Cole no Claude Code

CLAUDE CODE – COLE ESTE PROMPT

Crie o worker autônomo do JARVIS que opera em background gerando ideias de projetos. Crie:

1. `jarvis-worker.js` (raiz do projeto) – script Node.js puro sem dependências de framework que:

- Roda ciclos a cada 3 minutos via `setInterval`
- `IDEA_THEMES`: array com 10 temas (SaaS de produtividade, ferramenta IA, app mobile, automação, produto digital, etc.)
- `IDEA_AGENT_ROLES`: mapa dos 6 agentes AIOS Core com nome da seção e ângulo específico (`@analyst`=análise mercado, `@developer`=arquitetura técnica, `@researcher`=inteligência competitiva, `@writer`=copy e posicionamento, `@ux-design-expert`=UX e design, `@manager`=roadmap e lançamento)
- `runIdeaCycle()`: escolhe tema rotativo, roda os 6 agentes em `Promise.all` via `@anthropic-ai/sdk` (sem importar o SDK do Next.js), cada agente gera sua seção, junta tudo em um markdown completo, salva em Supabase como `jarvis_files` com `project_name` começando com "💡"
- `runAgentTask(agentId, theme, role)`: chama Claude Sonnet 4.6, system prompt específico do agente, retorna texto da seção
- Funções de sync com Supabase: `runCerebroSync()` e `runOraculoSync()` que chamam as APIs locais
- Tratamento de erros com `retry` e logs claros

2. Adicione ao `package.json`: `"worker": "node jarvis-worker.js"`

Rode `node jarvis-worker.js` e confirme que após 3 minutos aparece um novo arquivo "💡 ..." na aba DOCS.

RESULTADO ESPERADO

- ✓ Worker roda em terminal separado sem erros
- ✓ A cada 3 minutos: arquivo "💡 [Ideia] – Brief Completo.md" no DOCS
- ✓ Brief tem 6 seções dos agentes (análise, tech, UX, copy, etc.)

✓ Log no terminal mostra progresso de cada agente

Tela de Senha + Boot Sequence

Acesso protegido com estética Iron Man

► COMANDO 11 – Cole no Claude Code

CLAUDE CODE – COLE ESTE PROMPT

Adicione a tela de autenticação e sequência de boot estilo Iron Man.
Crie:

1. `src/components/cockpit/PasswordGate.tsx` – tela de senha que:
 - Fundo completamente preto com partículas ou hex grid
 - Logo "J.A.R.V.I.S" em Orbitron grande com glow azul
 - Campo de senha com borda cyan, placeholder "CÓDIGO DE ACESSO"
 - Senha correta: "1234" (configurável) – ao acertar, fade out e avança
 - Senha errada: shake animation e mensagem "ACESSO NEGADO"
 - Efeito de digitação com cursor piscando
2. `src/components/cockpit/BootSequence.tsx` – sequência de inicialização que:
 - Mostra linhas de texto aparecendo uma a uma (efeito typewriter)
 - Textos: "INICIALIZANDO J.A.R.V.I.S 3.0...", "CARREGANDO MÓDULOS DE IA...", "CONECTANDO SUPABASE...", "AGENTES ONLINE...", "SISTEMAS PRONTOS."
 - Barra de progresso cyan enchendo
 - Duração total ~3 segundos
 - Ao concluir, emite `onComplete()`
3. `src/components/cockpit/MicPermissionOverlay.tsx` – tela pedindo permissão do microfone antes do boot
4. Integre no `JarvisCockpit` com estados: `'locked' → 'mic-prompt' → 'booting' → 'ready'`

A senha padrão pode ser "1234". Documente como mudar no código.

RESULTADO ESPERADO

- ✓ Ao abrir o site: tela preta com campo de senha
- ✓ Digite a senha → boot sequence com textos aparecendo
- ✓ Após boot: cockpit completo fica visível com fade in

✓ Senha errada: animação de erro vermelha

PARTE 3 – PASSO 14

Publicar no GitHub

Repositório remoto para o deploy automático funcionar

◆ ANTES DE CONTINUAR

Crie um repositório novo no github.com (sem README, sem .gitignore). Copie a URL do repositório (ex: `https://github.com/SEU-USUARIO/meu-jarvis.git`)

► COMANDO 12 – Cole no Claude Code (substitua a URL)

CLAUDE CODE – COLE ESTE PROMPT

```
Configure o repositório remoto e faça o primeiro push. Execute:  
git remote add origin https://github.com/SEU-USUARIO/meu-jarvis.git  
git add -A  
git commit -m "feat: JARVIS AIOS completo – voz, agentes, memória,  
DOCS"  
git branch -M main  
git push -u origin main
```

Confirme que o push foi bem-sucedido e mostre o link do repositório.

RESULTADO ESPERADO

- ✓ Código aparece no seu repositório GitHub
- ✓ Todos os arquivos visíveis no github.com
- ✓ .env.local NÃO aparece (está no .gitignore)

Deploy na Vercel – JARVIS Online

Do código ao ar em 5 minutos, gratuitamente

1 Acesse vercel.com e faça login com GitHub

Clique em "New Project" → "Import Git Repository" → selecione seu repositório meu-jarvis

2 Configure as variáveis de ambiente

Na tela de configuração, clique em "Environment Variables" e adicione TODAS as chaves do seu .env.local (ANTHROPIC_API_KEY, OPENAI_API_KEY, NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY e as demais)

3 Clique em Deploy

A Vercel vai buildar e publicar. Em ~2 minutos você receberá uma URL: seu-projeto.vercel.app

► COMANDO 13 – Cole no Claude Code

CLAUDE CODE – COLE ESTE PROMPT

```
Crie o arquivo vercel.json na raiz do projeto com a configuração correta para o JARVIS:  
- framework: "nextjs"  
- functions com maxDuration 60s para: jarvis-chat, agent-execute, oraculo/cycle, cerebro  
- cron job para /api/oraculo/cycle rodando às 6h todos os dias  
Depois faça commit e push para o GitHub acionar o redeploy automático.
```

RESULTADO ESPERADO

- ✓ JARVIS acessível em <https://seu-projeto.vercel.app>
- ✓ Tela de senha funcionando
- ✓ Chat com Claude funcionando
- ✓ Deploy futuro: só git push → Vercel atualiza em 30s

► DEPLOY AUTOMÁTICO CONFIGURADO

Qualquer mudança que você pedir para Claude fazer e depois executar `git add -A && git commit -m "..."` && `git push` → Vercel redeploys automaticamente em ~30 segundos.

Como Falar com o JARVIS

Guia rápido de uso diário

Wake Word – Ativação por voz

VOCÊ FALA: "JARVIS, [seu pedido]" | Wake word detectado | Conversation mode ativo por 30 segundos |  | Resposta de voz
Follow-up livre (JARVIS fala) (sem precisar dizer JARVIS nos próximos 30 segundos)

Exemplos de comandos de voz

VOCÊ DIZ	JARVIS FAZ
"JARVIS, qual é a hora?"	Responde com a hora atual
"JARVIS, guarde que meu projeto atual é X"	Chama remember_fact → salva no Supabase
"JARVIS, crie um script Python para renomear arquivos"	Chama write_file → aparece no DOCS
"JARVIS, crie um site completo de portfólio"	Chama save_project → ZIP no DOCS
"JARVIS, o que você lembra sobre meus projetos?"	Chama search_memory → recupera contexto

Botão de microfone



MIC ATIVO (verde/cyan)

Microfone ligado. JARVIS está ouvindo passivamente. Diga "JARVIS" para ativar.



MIC MUDO (vermelho)

Microfone desligado. JARVIS não ouve nada. Clique para reativar.

Como Pedir Projetos e Expansões

Comandos para melhorar e expandir seu JARVIS via Claude Code

Pedindo novas funcionalidades

Sempre que quiser adicionar algo novo, entre na pasta do projeto, execute

`claude` e use prompts como estes:

► EXEMPLO - Adicionar novo agente

CLAUDE CODE - COLE ESTE PROMPT

Adicione um novo agente chamado @contador ao sistema AIOS. Ele é especialista em contabilidade brasileira, MEI, simples nacional e planejamento tributário. Adicione seu system prompt no /api/agent-execute e mencione-o no jarvisPrompt.ts para o JARVIS saber quando delegar para ele.

► EXEMPLO - Solicitar projeto completo

CLAUDE CODE - COLE ESTE PROMPT

No JARVIS, diga: "JARVIS, crie um projeto completo de landing page para venda de curso online com HTML, CSS e JavaScript. Inclua seção hero, benefícios, depoimentos, preço e formulário de contato."

► EXEMPLO - Atualizar visual do cockpit

CLAUDE CODE - COLE ESTE PROMPT

Atualize a interface do cockpit adicionando um widget de notícias de IA no canto superior direito do centro. Deve buscar do ORÁCULO as últimas 3 entradas de oraculo_knowledge e exibir como cards pequenos com título e snippet, atualizando a cada 60 segundos.

Comandos de manutenção

```
# Rodar localmente
npm run dev

# Rodar o worker autônomo (terminal separado)
node jarvis-worker.js

# Verificar erros TypeScript
npx tsc --noEmit
```

```
# Deploy (após mudanças)
```

```
git add -A && git commit -m "feat: sua descrição" && git push
```

Checklist Final + Referência Rápida

Confirme que tudo está funcionando

Checklist de implementação

- ☐ Claude Code instalado e autenticado
- ☐ Contas Anthropic + OpenAI + Supabase + Vercel criadas
- ☐ Projeto Next.js criado com todas as dependências
- ☐ Schema SQL executado no Supabase (5 tabelas)
- ☐ .env.local com todas as chaves configuradas
- ☐ Interface visual Iron Man funcionando em localhost:3000
- ☐ Chat com Claude respondendo em streaming
- ☐ Voz funcionando — "JARVIS" ativa, fala sem chamar não ativa
- ☐ Memória persistindo após reload da página
- ☐ 21 agentes visíveis no painel lateral
- ☐ Painel DOCS com botões de download funcionando
- ☐ Worker gerando ideias a cada 3 minutos
- ☐ Tela de senha e boot sequence funcionando
- ☐ Código no GitHub
- ☐ Deploy na Vercel funcionando online

Referência de variáveis de ambiente

VARIÁVEL	ONDE PEGAR	OBRIGATÓRIA?
ANTHROPIC_API_KEY	console.anthropic.com → API Keys	Sim
OPENAI_API_KEY	platform.openai.com → API Keys	Sim (voz)
NEXT_PUBLIC_SUPABASE_URL	supabase.com → Settings → API	Sim
NEXT_PUBLIC_SUPABASE_ANON_KEY	supabase.com → Settings → API	Sim
NEXT_PUBLIC_WAKE_WORD	Defina como: jarvis	Sim
NEXT_PUBLIC_JARVIS_VOICE	alloy / echo / fable / onyx / nova	Sim
GEMINI_API_KEY	aistudio.google.com (gratuito)	Não
GROQ_API_KEY	console.groq.com (gratuito)	Não

Custo estimado mensal

SERVIÇO	USO PESSOAL (1-2h/dia)	USO INTENSO (8h/dia)
Anthropic (Claude)	~\$5-10 USD	~\$30-60 USD
OpenAI (Voz Realtime)	~\$3-8 USD	~\$20-40 USD
Supabase	Gratuito	Gratuito
Vercel	Gratuito	Gratuito
TOTAL	~\$8-18 USD/mês	~\$50-100 USD/mês

► SEU JARVIS ESTÁ PRONTO

Você acabou de construir uma IA pessoal com voz, 21 agentes autônomos, memória permanente e deploy em produção — sem escrever uma linha de código manualmente. **Claude fez tudo por você.** Agora qualquer melhoria futura é só um comando.

O que é o AIOS CORE e para que serve

O núcleo de inteligência que comanda todo o sistema

AIOS CORE (AI Operating System Core) é o conjunto dos **6 agentes de propósito geral** que formam o núcleo intelectual do JARVIS. Eles são o equivalente a ter uma equipe completa de especialistas disponível 24 horas — analista, desenvolvedor, pesquisador, escritor, designer e gerente de projetos — todos coordenados pelo JARVIS e acionados por comando de voz ou chat.

SEM AIOS CORE COM AIOS CORE

— Você faz uma pergunta Você dá um objetivo

JARVIS responde com texto JARVIS delega para os agentes certos Você pesquisa, escreve, Agentes pesquisam, escrevem, analisa e decide sozinho analisam e entregam — você aprova Resultado: você como executor Resultado: você como CEO

Por que o AIOS CORE é diferente de um chatbot

CHATBOT COMUM	AIOS CORE
Responde perguntas	Executa tarefas complexas de ponta a ponta
Um modelo genérico	6 especialistas com system prompts calibrados
Sem memória persistente	Aprende e acumula conhecimento no banco
Trabalho sequencial	Todos os agentes em paralelo simultaneamente
Sem entregáveis	Arquivos reais entregues na aba DOCS
Sem autonomia	Ciclos automáticos de ideias e produção

A hierarquia do sistema

VOCÊ | JARVIS ← orquestrador central (voz + chat) |

| | | AIOS CORE FÁBRICA ORÁCULO (6 agentes) (15

agentes) (pesquisa) Intelecto Produção Saber geral e vendas contínuo | | |
| | | | SUPABASE (memória) DOCS (entregas)

Instalar o AIOS CORE do Zero

3 comandos – Claude configura tudo automaticamente

◆ PRÉ-REQUISITO

Você precisa ter o Claude Code instalado (`npm install -g @anthropic-ai/claude-code`) e uma pasta de projeto criada. Se já seguiu os Passos 01 a 05 deste guia, o AIOS CORE já está instalado como parte do JARVIS. Use estes comandos para instalar em um projeto separado ou do zero.

► COMANDO 14 – Criar projeto AIOS CORE standalone

CLAUDE CODE – COLE ESTE PROMPT

Crie um projeto Node.js chamado "meu-aios-core" com a seguinte estrutura:

- package.json com dependências: @anthropic-ai/sdk, @supabase/supabase-js, dotenv
 - .env com placeholders para: ANTHROPIC_API_KEY, NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY
 - src/agents/ com um arquivo para cada agente do AIOS CORE: analyst.js, developer.js, researcher.js, writer.js, ux-designer.js, manager.js
 - src/core/orchestrator.js que importa todos os agentes e expõe runAgents(task, agents[])
 - src/core/memory.js que conecta ao Supabase e salva/busca resultados
 - index.js como ponto de entrada com menu interativo para escolher qual agente acionar
- Instale as dependências e mostre a estrutura criada.

RESULTADO

- ✓ Pasta meu-aios-core/ com estrutura completa
- ✓ 6 arquivos de agente com system prompts individuais
- ✓ Orquestrador com execução em paralelo (Promise.all)
- ✓ Memória conectada ao Supabase

Preencha cada arquivo de agente em `src/agents/` com system prompts especializados e de alta qualidade:

@analyst: especialista em análise de mercado, tendências, dados competitivos, ROI, tomada de decisão estratégica baseada em evidências. Entrega relatórios estruturados com dados, insights acionáveis e recomendações concretas.

@developer: engenheiro sênior full-stack especializado em TypeScript, Node.js, Python, React, APIs REST, banco de dados. Entrega código completo, funcional, comentado, nunca truncado ou com placeholders.

@researcher: especialista em pesquisa profunda, validação de dados, benchmarking, curadoria de fontes primárias. Entrega relatórios com fontes citadas, comparativos e sínteses objetivas.

@writer: copywriter e redator especializado em copy de alto impacto, conteúdo digital, roteiros, e-mails de conversão, posts virais. Entrega textos prontos para publicar, com CTAs e estrutura persuasiva.

@ux-designer: especialista em UX/UI, arquitetura de informação, wireframes descritivos, experiência do usuário, acessibilidade. Entrega especificações visuais, fluxos de usuário e protótipos em texto/ASCII.

@manager: especialista em gestão de projetos, metodologias ágeis, OKRs, roadmaps, priorização por impacto. Entrega planos executáveis, cronogramas, matrizes de prioridade e estruturas de projeto.

Cada agente deve receber contexto do usuário + histórico do Supabase antes de responder. `max_tokens: 8192` para todos.

► COMANDO 16 – Criar interface de linha de comando para usar o AIOS CORE

CLAUDE CODE – COLE ESTE PROMPT

Crie uma interface CLI interativa no `index.js` do projeto `meu-aios-core` que permita:

1. Digitar uma tarefa livremente (ex: "analise o mercado de SaaS para pequenas empresas")
2. Escolher quais agentes ativar: todos, ou selecionar individualmente
3. Ver o progresso de cada agente em tempo real no terminal
4. Receber os resultados consolidados em um único relatório markdown
5. Salvar automaticamente o resultado em uma pasta "entregas/" com timestamp
6. Opção de salvar no Supabase para histórico persistente

Use `readline` para o input interativo. Mostre os resultados com formatação colorida no terminal (use cores ANSI: verde para sucesso, cyan para progresso, amarelo para avisos).

COMO USAR APÓS INSTALAR

- ✓ Configure seu `ANTHROPIC_API_KEY` no arquivo `.env`
- ✓ Execute: `node index.js`
- ✓ Digite sua tarefa e escolha os agentes
- ✓ Relatório salvo em `entregas/[timestamp]-resultado.md`

Os 6 Agentes do AIOS CORE – O que cada um faz

Especialistas autônomos – cada um com uma área de domínio

@analyst – Estratégia e Inteligência de Mercado



O que ele faz

Analisa mercados, concorrentes, tendências e oportunidades. Transforma dados brutos em decisões estratégicas. Calcula ROI, avalia riscos, valida hipóteses de negócio com evidências.

PEÇA ISSO	ELE ENTREGA
"Analise o mercado de ferramentas IA para RH"	Relatório com tamanho de mercado, players, gaps e oportunidade
"Avalie se minha ideia X tem potencial"	Análise SWOT, validação de demanda, projeção de receita
"Quais são os 5 maiores erros em lançamentos SaaS?"	Análise baseada em casos reais com lições aprendidas
"Compare os modelos de precificação mais usados"	Tabela comparativa com prós/contras e recomendação

@developer – Engenharia de Software



O que ele faz

Escreve código completo, funcional e pronto para usar. Cria APIs, scripts, automações, aplicações web, integrações. Nunca entrega código truncado ou com placeholders — é o arquivo real.

PEÇA ISSO	ELE ENTREGA
"Crie uma API REST de cadastro de usuários"	server.js completo com rotas, validação, JWT, banco
"Script Python para automatizar relatórios Excel"	Script funcional com pandas, pronto para rodar
"Integração com API do WhatsApp Business"	Código completo com webhooks e envio de mensagens
"Crie um scraper para coletar preços do site X"	Script com tratamento de erros e exportação CSV

@researcher – Pesquisa Profunda



O que ele faz

Pesquisa com profundidade e rigor. Valida informações, cita fontes, compara perspectivas. Ideal para due diligence, benchmarking, curadoria de conteúdo e inteligência competitiva.

PEÇA ISSO	ELE ENTREGA
"Quais são as melhores ferramentas gratuitas de automação?"	Lista curada com comparativo detalhado e casos de uso
"Pesquise tendências de IA em 2025"	Relatório com fontes, dados e análise de impacto
"Faça um benchmark dos meus concorrentes"	Tabela comparativa de features, preços e posicionamento
"Quais nichos de produtos digitais têm menos concorrência?"	Análise de oportunidades com critérios de validação

@writer – Conteúdo e Copywriting



O que ele faz

Escreve textos de alto impacto para qualquer formato. Copy de vendas, e-mails, posts, roteiros, e-books, scripts de vídeo. Sempre com estrutura persuasiva, CTA claro e adaptado ao público.

PEÇA ISSO	ELE ENTREGA
"Escreva uma landing page para meu SaaS"	Copy completo: headline, benefícios, prova social, CTA
"Crie uma sequência de 5 e-mails de onboarding"	5 e-mails completos com assunto, corpo e CTA
"Roteiro de vídeo de 60 segundos para TikTok"	Roteiro com gancho, desenvolvimento e chamada para ação
"E-book de 20 páginas sobre produtividade com IA"	Estrutura completa com capítulos, exemplos e exercícios

@ux-design-expert – UX e Design



O que ele faz

Projeta experiências de usuário completas. Wireframes em texto/ASCII, fluxos de navegação, especificações de interface, guias de estilo, análise de usabilidade e recomendações de design.

PEÇA ISSO	ELE ENTREGA
"Projete o fluxo de onboarding do meu app"	Fluxo passo a passo com telas descritas e decisões UX
"Crie wireframe do dashboard do meu SaaS"	Layout ASCII detalhado com hierarquia visual explicada
"Analise a usabilidade do meu checkout"	Pontos de fricção, sugestões de melhoria, benchmark
"Defina a paleta de cores e tipografia para minha marca"	Guia de estilo com hex codes, fontes e regras de uso

@manager – Gestão e Estratégia Operacional



O que ele faz

Organiza, prioriza e cria planos de execução. Roadmaps, sprints, OKRs, matriz de prioridades, cronogramas, estruturas de projeto. Transforma ideias em planos acionáveis com responsáveis e prazos.

PEÇA ISSO

ELE ENTREGA

"Crie um roadmap de 90 dias para lançar meu produto"

Roadmap detalhado por semana com milestones e riscos

"Quais são as tarefas prioritárias para lançar um SaaS?"

Lista priorizada por impacto x esforço com justificativas

"Monte a estrutura de um sprint de 2 semanas"

Sprint backlog, daily check-in template, critérios de done

"Como estruturo uma equipe de 3 pessoas para escalar?"

Organograma, responsabilidades, processos e rituais

O que você pode construir com o AIOS CORE

Casos de uso reais - do pessoal ao profissional

Para uso pessoal



Assistente de decisões pessoais

"JARVIS, analise os prós e contras de eu mudar de carreira para IA" → @analyst + @researcher entregam análise completa de mercado de trabalho, cursos, salários e caminho sugerido.



Aprendizado acelerado

"JARVIS, crie um plano de estudos de 30 dias para aprender Python do zero" → @manager faz o cronograma, @researcher curada os melhores recursos, @developer cria os exercícios práticos.



Produção de conteúdo pessoal

"JARVIS, escreva meu currículo completo para a área de dados" → @writer redige, @analyst posiciona, @ux-designer formata.

Para freelancers e criadores



Propostas comerciais em minutos

"JARVIS, crie uma proposta completa para um projeto de automação para um cliente de e-commerce" → todos os agentes colaboram: análise, escopo técnico, copy persuasivo, estimativa de horas e preço.



Produtos digitais automatizados

A Fábrica Rentável (15 agentes) opera em conjunto com o AIOS CORE: você diz o nicho, eles pesquisam, criam, empacotem e publicam o produto digital nos marketplaces.



Presença digital completa

"JARVIS, crie minha estratégia de conteúdo para os próximos 30 dias no Instagram e LinkedIn" → @researcher analisa tendências, @writer cria os posts, @manager monta o calendário editorial.

Para empreendedores e startups



MVP em horas, não meses

"JARVIS, crie o MVP completo de um SaaS de agendamento para salões de beleza" → @developer escreve o código, @ux-designer projeta as telas, @writer cria a landing page, @manager define o roadmap de lançamento.



Inteligência de negócio contínua

O ORÁCULO pesquisa automaticamente todos os dias o que há de novo no seu setor. @analyst processa esses dados e você recebe um briefing diário sem fazer nada.



Operação autônoma da Fábrica Rentável

Configure o worker e a Fábrica opera sozinha: pesquisa nichos → cria produtos → publica → divulga → atende clientes → transfere receita. O AIOS CORE supervisiona a qualidade em cada etapa.

Exemplos de sessões reais de AIOS CORE em paralelo

VOCÊ: "JARVIS, quero lançar um curso sobre automação com IA" | JARVIS delega para TODOS em paralelo: | — @analyst → Analisa mercado: tamanho, concorrência, preço ideal | — @researcher → Pesquisa melhores cursos existentes e gaps de conteúdo | — @writer → Cria copy da landing page e sequência de e-mails | — @developer → Cria página de vendas em HTML completa | — @ux-designer → Projeta estrutura do curso e wireframe da plataforma | — @manager → Monta cronograma de produção e plano de lançamento | Em ~5 minutos: 6 documentos completos na aba DOCS Você leu, aprovou e começou a executar.

Expandir com Novos Agentes Customizados

O AIOS cresce com você – adicione qualquer especialidade

O AIOS CORE foi projetado para ser expandido. Você pode adicionar agentes para qualquer área de domínio — e Claude faz tudo por você com um único comando.

► COMANDO 17 – Adicionar um agente especializado

CLAUDE CODE – COLE ESTE PROMPT

Adicione um novo agente ao sistema AIOS chamado @juridico. Ele é especialista em direito digital brasileiro, contratos SaaS, LGPD e termos de uso. Deve:

- Ter seu próprio system prompt completo em src/agents/juridico.js (ou no dicionário AGENTS do agent-execute)
- Ser mencionado no jarvisPrompt.ts para que o JARVIS saiba quando delegar para ele
- Responder perguntas sobre: análise de contratos, adequação LGPD, termos de uso e políticas de privacidade
- Entregar documentos estruturados com seções claras, alertas de risco e recomendações acionáveis

Após criar o agente, crie um exemplo de uso: peça para ele gerar um modelo de Termos de Uso para um SaaS brasileiro e confirme que a entrega aparece no painel DOCS.

OUTROS AGENTES QUE VOCÊ PODE CRIAR

- ✓ @contador – contabilidade brasileira, MEI, simples nacional, DRE
- ✓ @marketing-digital – Google Ads, Meta Ads, funis de conversão
- ✓ @rh – descrições de cargo, processos seletivos, onboarding
- ✓ @suporte – base de conhecimento, FAQs, scripts de atendimento
- ✓ @data-scientist – análise de dados, SQL, visualizações, insights
- ✓ @seo – palavras-chave, meta tags, estratégia de conteúdo orgânico

► COMANDO 18 – Criar um agente especializado no seu negócio

CLAUDE CODE – COLE ESTE PROMPT

Crie um agente personalizado chamado @especialista-[seu-nicho] treinado especificamente para o meu negócio. O contexto do negócio é: [descreva seu negócio, produto, público-alvo e principais desafios]. O agente deve conhecer profundamente: [liste as áreas de conhecimento específicas]. Ele deve ser capaz de: [liste os tipos de tarefa que vai executar]. Crie o system prompt completo, adicione ao sistema de agentes e mostre um exemplo de uso prático.

► DICA DE OURO

Quanto mais contexto você der ao Claude sobre o seu negócio, mais preciso e útil será o agente criado. Inclua: nicho, público-alvo, linguagem da marca, produtos/serviços, casos de uso frequentes e exemplos do tom desejado.

ORÁCULO + CÉREBRO – Inteligência Contínua

O sistema que aprende e evolui sozinho todos os dias

O ORÁCULO – Pesquisa autônoma permanente

O ORÁCULO é um agente especial Level 7 que roda em ciclos automáticos. Ele pesquisa o que há de mais novo em IA, tecnologia e mercado e salva no banco de dados. Com o tempo, o JARVIS fica progressivamente mais informado **sem você fazer nada**.

```
CICLO DO ORÁCULO (automático 1x/dia na Vercel + 1x/min local) | GET
/api/oraculo/cycle | Claude @oraculo analisa tópicos da base + tendências
atuais | Gera novo insight sobre: IA generativa · ferramentas · mercado ·
tech | Salva em oraculo_knowledge (Supabase) | Disponível para TODOS os
agentes via query_oraculo | JARVIS fica mais inteligente a cada ciclo
```

► COMANDO 19 – Configurar e expandir o ORÁCULO

CLAUDE CODE – COLE ESTE PROMPT

```
Configure o ORÁCULO para pesquisar sobre os tópicos mais relevantes
para o meu negócio. Adicione os seguintes temas de pesquisa ao array
ORACULO_TOPICS no arquivo src/app/api/oraculo/route.ts (ou
equivalente): tendências de IA para [meu setor], ferramentas e
automações para [meu nicho], cases de sucesso de [tipo de negócio],
novidades em modelos de linguagem e suas aplicações práticas,
oportunidades de mercado em [área de interesse].
Também configure o ORÁCULO para salvar um resumo semanal consolidado
em um arquivo "briefing-semanal.md" na aba DOCS toda segunda-feira às
8h via cron na Vercel.
```

O CÉREBRO – Multi-IA para máxima qualidade

O **CÉREBRO** é o motor de síntese que combina múltiplas IAs em uma única resposta. Quando ativado, envia a mesma tarefa para Claude, Gemini e Llama simultaneamente e sintetiza o melhor de cada resposta — resultando em qualidade muito superior a qualquer modelo sozinho.

IA	PONTO FORTE	CHAVE NECESSÁRIA
Claude Sonnet 4.6	Raciocínio, código, análise longa	ANTHROPIC_API_KEY (pago)
Gemini 2.0 Flash	Velocidade, criatividade, multimodal	GEMINI_API_KEY (gratuito)
Llama 3.3 (Groq)	Ultra-rápido, open source, custo zero	GROQ_API_KEY (gratuito)

► COMANDO 20 – Ativar o CÉREBRO multi-IA

CLAUDE CODE – COLE ESTE PROMPT

Configure o sistema CÉREBRO no endpoint `/api/cerebro/route.ts` para aceitar uma tarefa e enviá-la em paralelo para Claude Sonnet 4.6 (usando `ANTHROPIC_API_KEY`), Gemini Flash (usando `GEMINI_API_KEY` via `fetch` para a API do Google AI) e Llama 3.3 via Groq (usando `GROQ_API_KEY` via cliente OpenAI apontando para `api.groq.com/openai/v1`). Após receber os 3 resultados, um quarto agente Claude sintetiza o melhor de cada resposta em um documento coeso e superior. Adicione o botão "⚡ CÉREBRO" na interface do JARVIS que aciona este endpoint. Confirme adicionando `GEMINI_API_KEY` e `GROQ_API_KEY` ao `.env.local` (ambas gratuitas).

► RESULTADO FINAL DO SEU SISTEMA

Com ORÁCULO + CÉREBRO + AIOS CORE + Fábrica Rentável, você tem uma inteligência que **aprende todos os dias, raciocina com múltiplas IAs, executa com 21 especialistas em paralelo** e entrega resultados reais. É o sistema de IA pessoal mais completo que existe — e você construiu tudo com comandos para o Claude.

◆ J.A.R.V.I.5 · AIOS 3.0 TURBO · Construído com Claude Code ◆

Anthropic Claude Sonnet 4.6 + OpenAI Realtime API + Next.js 15 + Supabase + Vercel